

PROGETTO DI INGEGNERIA DEL SOFTWARE

Find & Charge

Piattaforma moderna per la gestione e prenotazione di colonnine di ricarica.

A.A. 2025/2026



FIND & CHARGE

Davide Mascheretti

PO & Backend

Tommaso Maistrello

Scrum Master & Frontend

Francesco Valenari

Development Team

Claudio Morgera

Development Team



IL PROBLEMA

Range Anxiety: L'incertezza di non trovare un posto di ricarica libero.

Abusi: Postazioni occupate abusivamente senza diritto.

Assenza di dati storici.



LA SOLUZIONE

Deterministica: Prenotazione atomica a slot rigidi (30m).

Fisica: Accesso validato da QR Code Dinamico.

Controllo centralizzato della colonnine.

Difficoltà e Mitigazioni

02

TECNICHE

Concorrenza DB: Gestione conflitti su Firebase (NoSQL Distribuito).

Vaadin e Leaflet: Curve di apprendimento

ORGANIZZATIVE

Timing: Sovrapposizione critica con sessioni d'esame.

Team: Coordinamento sincrono.

STRATEGIA DI RISOLUZIONE

Sprint 0

SETUP RIGOROSO

Adaptive

PLANNING FLESSIBILE

Adattamento della lunghezza degli sprint (10-20gg) in base al carico accademico esterno.

Paradigmi & Tech Stack

03

CORE LOGIC

Java 17 LTS

Spring Boot 3.2

OOP

Approccio MDA:

Dal modello (Papyrus) al codice.

Refactoring:

Reverse engineering per allineamento finale.

FRONTEND

Vaadin 24

Pure Java UI

CSS Custom

Leaflet JS

DATA

Firebase

Realtime NoSQL

TOOL UTILIZZATI

JUnit 5

EclEmma

PMD

Stan4J

Maven

Configuration Management

04

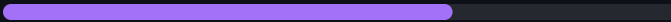


GitHub
Private Repository

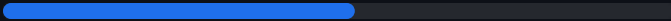
Commits: 250+



Issues: 35 Closed



Branches: 9 Active



WORKFLOW IBRIDO

Push diretto su main

Modifiche rapide, refactoring, docs.

Utilizzo di branch secondari

Funzionalità complesse, PR obbligatorie.

PROJECT MANAGEMENT



Kanban
Board

Gestione issues con labels (*enhancement*, *bug*).

Stati: **Todo** -> **In Progress** -> **Done**.

Software Life Cycle

05

MODELLO

Agile Scrum (Flessibile)

Sprint flessibili: **10 - 20 giorni**

Adattamento al carico esami



ELICITAZIONE DEI REQUISITI

- Brainstorming
- Analisi applicazioni concorrenti
- Simulazione scenari d'uso

PRIORITÀ MOSCOW

MUST HAVE

Autenticazione, Mappa, Prenotazione, QR Code

SHOULD HAVE

Profilo Utente, Dashboard Admin, Gestione Utenti

COULD HAVE

Recensioni, Geolocalizzazione, Email di conferma

REQUISITI NON FUNZIONALI



Performance

< 3 secondi



Sicurezza

BCrypt Hash



Concorrenza

No Overbooking



Affidabilità

Control Validation



Scalabilità

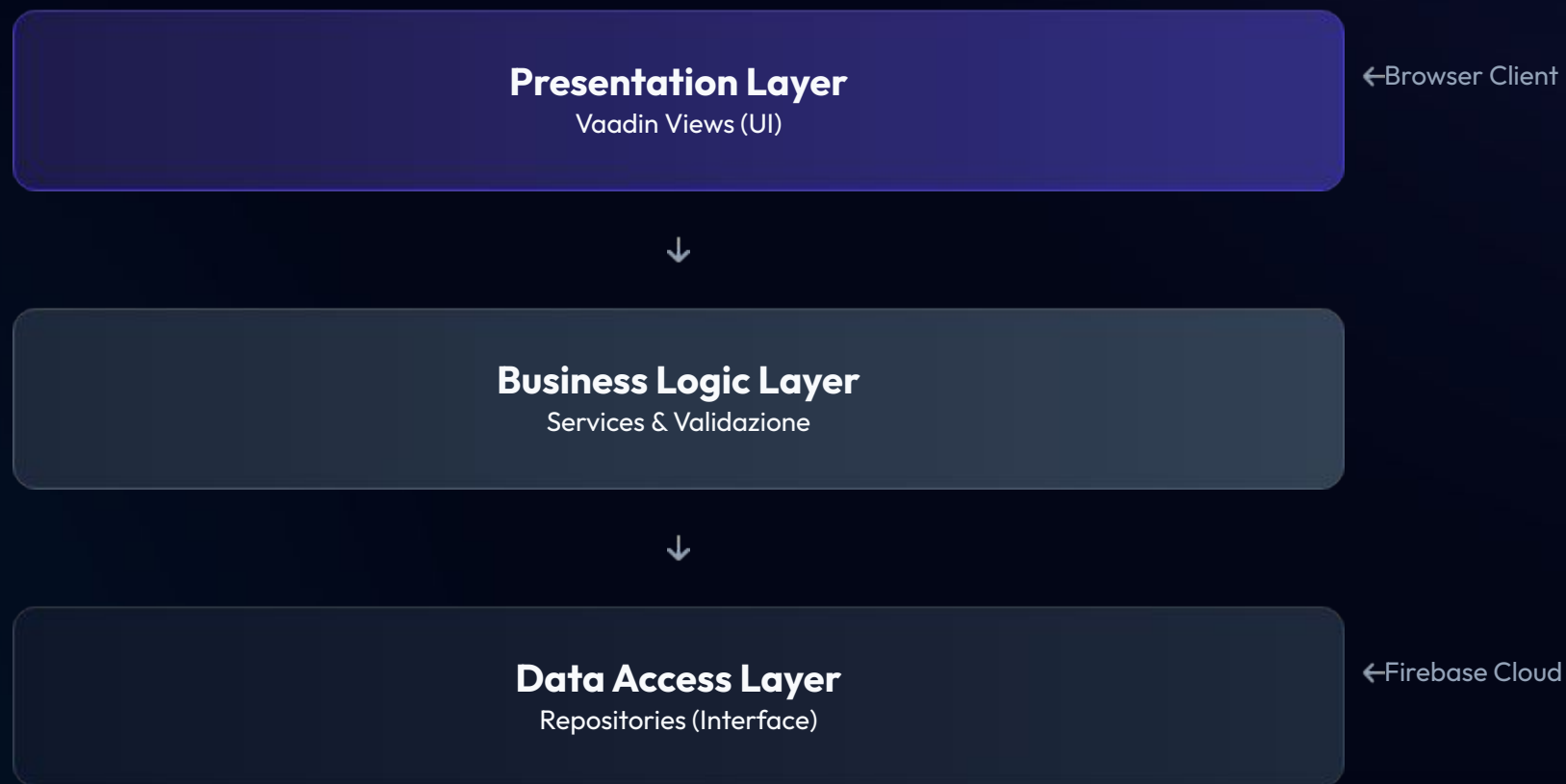
Supporto per un numero crescente di utenti e colonnine



Compatibilità

Accessibile da diversi dispositivi e browser

Layered Architecture: Massima separazione delle responsabilità.



PATTERNS

Singleton

Spring @Service e @Repository

Adapter

Database interface

Observer

Listeners ed eventi

METRICHE DEL CODICE (AGG. 01/02/2026)

3.505

ELOC

1,25

COMPLEXITY (CC)

2,27

COUPLING (CBO)

0,51

POLLUTION

Metriche di Martin

Distance (|D|) 0.37



Main Sequence

Struttura

Tangled 0%



No Cycles

DIAGRAMMI PRODOTTI

- Use Case Diagram
- State Machine Diagram
- Class Diagram
- Sequence Diagram
- Communication Diagram
- Activity Diagram
- Component Diagram
- Package Diagram

Esempio: Sequence Diagram (Prenotazione)



FEATURES

Auth & Security



Mappa Leaflet



Prenotazione Server-side



QR Generation



Pagamenti Reali



REFACTORING

- **Extract Class:**

Separazione logica DB dalle Viste (creazione *Services*).

- **Validazione dati centralizzata:**

Classe unica `DataValidator` per input e sicurezza.

- **Divisione parte Java e CSS:**

Gestione degli stili delegata a CSS esterni.

Testing & Coverage



STRATEGIA WHITE BOX

JUnit 5 per Unit Testing.

Focus su **Business Logic** e **Models**.

Esclusione consapevole delle viste Vaadin e del Database diretto.

Manual Testing

Functional Testing

COVERAGE REPORT (ECLEMMMA)

Auto Model 100%



TipoAuto (Enum) 96.4%



QRCode Generator 85.7%



DataValidator (Branch) 75%



Prenotazione 68.2%



LIVE DEMO



Esplorazione



Prenotazione



Admin